

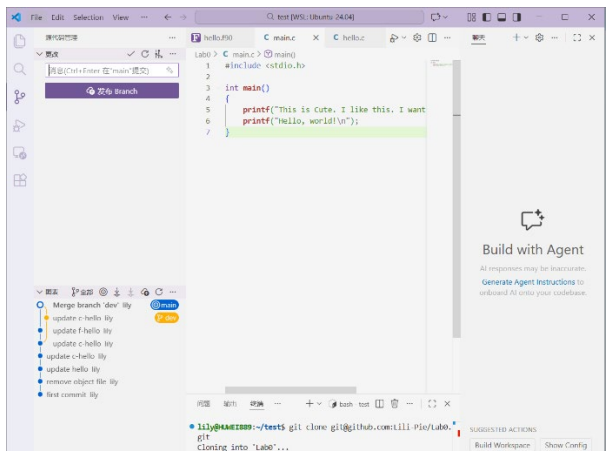
林志宏 25800190033

问题：

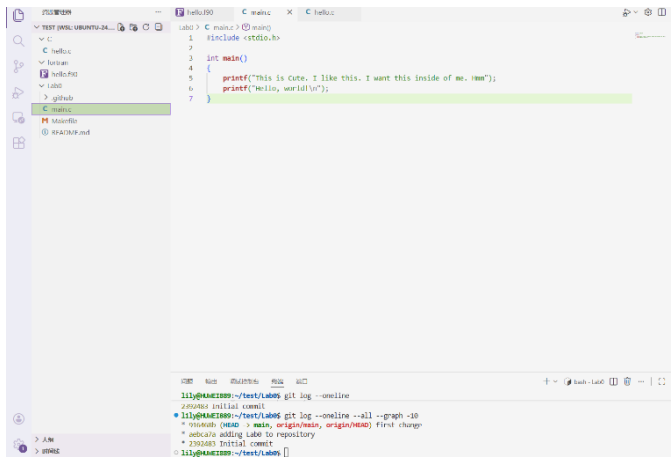
1. 没有软件/代码类的多人协同开发经历，但高中和同学一起维护过一个 Notion 上的知识库，用来整理拉丁语/心理学/社会学的各种知识。维护内容包括创建 Notion 的页面，页面结构优化，建立超链接关系网，检索机制等。大部分用 Notion 中的模板，还有一些是自己设计的页面，分类与检索方案。虽然不是代码开发，但多人维护知识库时也涉及内容分工、结构设计和协作规范。
2. 如果没有暂存区，用户难以在多个修改中选择部分内容提交。。有了暂存，提交只提交暂存区的内容，方便人筛选哪些需要提交，哪些不用，可以保证不同代码之间版本更新的独立性。同时暂存具有容错空间，可以取消暂存，提交前可以二次检查有无漏洞，是否需要提交。
3. git branch 只检查本地分支，git branch -a 检查本地和远程的跟踪分支（如 remote s/origin/main）。在多人协作时，git branch -a 可以用于检查是否有新的远程分支，便于确定本地分支对应哪个远程分支，便于同步代码。

实验步骤：

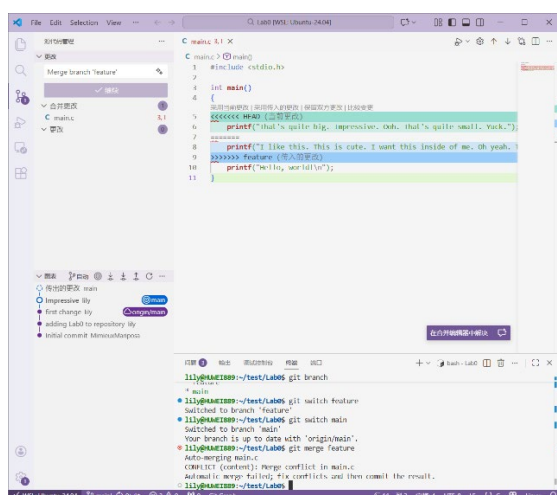
1. 使用模板创建个人仓库，并克隆到本地。



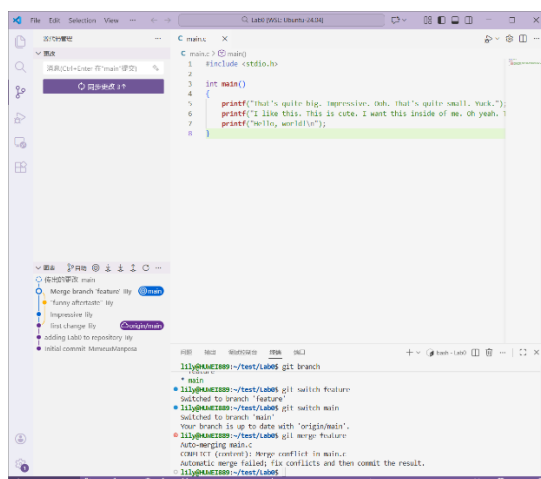
2. 对 main.c 进行了三次修改并提交



3. 创建了 feature 分支，在 main 与 feature 文件中同一个位置修改，导致合并冲突



4. 通过保留双方更改，解决冲突



网页阅读：

1. 《Commit Message 规范》简单讲解了 commit message 的作用，基本的格式（Header, Body, Footer 的写法，一些常用的标签分类），以及一些辅助撰写或检查提交消息的工具与指令。
2. 《Git Flow 分支控制》介绍了常用开发流程中 gitflow 的使用规范，包括 master, develop, feature, release, hotfix 几个分支，针对性讲解了各个分支的作用，分支间的联系，合并关系，以及基本的 git 分支操作代码。
3. 对于编程，尤其是大规模的项目编程以及多人协作，学习 git 极为重要。首先 git 可以保存版本，但和传统版本管理不同，git 只保存一次相同的文件，之后更改都只保存更改的内容，并且会进行压缩，减小存储压力。其次 git 的时间线管理可以方便同时进行多种更改，并保持互相独立，或是多人在不同机器上同时协作，负责不同的内容，最后还能一致合并，保证版本的前后连贯性。是现代开发的关键技术。对于生活，学习 git 可以了解应用与游戏每次更新的更新消息，如 feature 表示新特性，release 表示测试版等，也能区分每次更新的实际效果。